



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

دانشکده‌ی مهندسی کامپیوتر

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین اول

فهرست مسائل

۱		مسئله ۱
۱		الف) چسبندگی (Cohesion)
۱		ب) اصل LSP در SOLID
۲		مسئله ۲

پاسخ مسئله‌ی ۱.

الف) چسبندگی (Cohesion)

چسبندگی به میزان ارتباط و انسجام اجزای داخلی یک ماژول یا کلاس اشاره دارد. به عبارت دیگر، نشان می‌دهد که عناصر یک واحد نرم‌افزاری تا چه حد در راستای یک هدف مشخص و مشترک عمل می‌کنند.

در طراحی مناسب، هر کلاس یا ماژول باید یک مسئولیت مشخص و متمرکز داشته باشد. در این حالت، گفته می‌شود چسبندگی بالا است که یک ویژگی مطلوب محسوب می‌شود. در مقابل، اگر یک کلاس چندین وظیفه نامرتبط را بر عهده داشته باشد، چسبندگی آن پایین بوده و این موضوع باعث کاهش خوانایی، نگهداری‌پذیری و قابلیت توسعه سیستم می‌شود.

جمع‌بندی: چسبندگی بالا به معنای تمرکز هر واحد نرم‌افزاری بر یک هدف مشخص و واحد است.

ب) اصل LSP در SOLID

اصل جایگزینی لیسکوف بیان می‌کند که اشیاء یک کلاس پایه باید بتوانند بدون ایجاد اختلال در درستی برنامه، با اشیاء کلاس‌های مشتق‌شده جایگزین شوند.

به بیان دقیق‌تر، کلاس‌های فرزند باید رفتارهایی سازگار با قرارداد تعریف‌شده در کلاس پایه ارائه دهند و نباید پیش‌شرط‌ها را سخت‌تر یا پس‌شرط‌ها را ضعیف‌تر کنند. در غیر این صورت، استفاده از Polymorphism منجر به بروز خطاهای منطقی در سیستم خواهد شد.

مثال معروف: فرض کنید کلاس Rectangle را داریم. اگر بخوایم کلاس Square را بسازیم و از Rectangle ارث ببرد، به مشکل می‌خوریم! چون اگر عرض را تغییر دهیم، طول هم باید تغییر کند. اما Rectangle انتظار دارد که ما می‌توانیم عرض و طول را جداگانه عوض کنیم. پس اگر جایی از کد Rectangle استفاده می‌کنیم و ما به جای آن Square بزاریم، منطق و کد به مشکل می‌خورد.

پاسخ مسئله‌ی ۲.

خیر، لزوماً این‌طور نیست.

متخصص شدن در یک حوزه‌ی خاص از مهندسی نرم‌افزار (مثلاً فقط پایگاه‌داده، فرانت‌اند یا الگوریتم‌ها) به این معنا نیست که آن فرد حتماً معمار نرم‌افزار خوبی هم خواهد شد. معماری نرم‌افزار نیازمند دیدی گسترده‌تر از صرفاً تسلط بر یک موضوع است.

یک معمار خوب باید بتواند بین بخش‌های مختلف سیستم ارتباط برقرار کند، تصمیم‌های کلان بگیرد، و trade-off ها (مثل کارایی، مقیاس‌پذیری، نگهداری‌پذیری و امنیت) را به‌درستی مدیریت کند. این مهارت‌ها معمولاً از تجربه در حوزه‌های مختلف، درک کلی از سیستم‌ها، و توانایی تحلیل مسائل در سطح بالا به دست می‌آیند، نه فقط از عمیق شدن در یک زمینه‌ی خاص.

بنابراین، تخصص عمیق در یک حوزه می‌تواند یک مزیت باشد، اما برای تبدیل شدن به یک معمار خوب کافی نیست؛ بلکه باید در کنار آن، دید سیستمی و تجربه‌ی گسترده نیز وجود داشته باشد.